# Exam project - Drone simulation

Geir Tummasarson
John Askham
Magnus Hansen

Object Oriented Programming with Java 1
Lesandi B.Sc. í KT-verkføði
Fróðskaparsetur Føroya

31-04-2024

FRÓÐSKAPARSETUR
FØROYA

# Contents

**1 Introduction** **1**

**2 Summary of the project** **1**

**3 Solution** **1**

3.1 Structure of the main loop . . . . . 2

3.2 Use of Inheritance . . . . . 2

3.3 The crime objects . . . . . 4

**4 Weaknesses of our solution** **4**

**5 Conclusion** **5**

# 1 Introduction

This report outlines the end of course exam project we created for the course Object Oriented Programming with Java 1. The project is aimed to make us show our abilities to work with objects and to show our understanding in how they interact with each other.

The platform of choice for developing our project was Windows 11 and macOS, alongside the development tool used being Visual Studio Code and JavaSE-21.

# 2 Summary of the project

The point of the project is to make a simulation of delivery drones that deliver products for a company called AmaZonaZZ.

The simulation assigns an order to a delivery drone, the delivery drone then heads to a warehouse to pick up the ordered item and delivers it to a specified location within the order, then heads back to the headquarter to await another order. When delivering the order, criminal events may happen, and how the drones respond to this criminal event depends on what type of drone it is, with there being two types of drones, high speed drones, and armored drones.

The simulation's standard time frame is 10:00 to 19:00, but we have changed it so that the user can specify the start and end point of the day. The end point does not signify when the simulation ends, but when new orders stop getting assigned, and the simulation then ends when all drones are back at the headquarter or neutralized by criminal events.

Aside from this, the simulation keeps track of all kinds of statistics, such as what drones are neutralized, what their current status is, revenue made by the company, operational costs, and much more. The most relevant of these is then printed out at the end of the simulation, giving the user an overview of what happened over the course of the simulation.

# 3 Solution

This section is to loosely explain the structure of the project, the different responsibilities of each class, and how everything works together to bring the simulation to life.

## 3.1 Structure of the main loop

The class which ties everything together is called `Simulation`, this is where the main loop runs and where all the set up is done. The simSetup method reads in all the data from the given txt files, splits it into usable data with regex, and returns generic ArrayLists containing the objects, such as drones, orders, items, and so on. After the setup, the main loop method is called and prompts the user for a start and end point, leading into the main loop.

Our design for the main loop consists of a check that keeps going as long as the end time is not reached yet, and all drones are either docked or neutralized. First thing the user interacts with in the main loop is the menu, the menu is a switch case, where the user is prompted for an input to see some form of statistic connected to the drones or simulation itself, and if left blank or the matches with nothing in the menu, the default case is chosen, and moves onto the next iteration of the loop.

All the drones are then looped through, and by use of a switch case and the drone state as a switch expression, the drones can be given different commands according to what their current state is. Our design differed slightly from the given problem statement, where instead of one `IN_TRANSIT` state, we have three different ones, controlling whether the drone is on its way to a warehouse, the order location or back to the headquarter. We figured this was a better and more readable design choice, instead of using boolean variables to control where a drone was on its way to.

Every half an hour the warehouses are restocked, every hour new drones are created with stats identical to those of the neutralized drones, each neutralized drone is only created once, so as to not have multiple copies be created of each drone. Each loop every drone is also checked if it has collided with another drone, and whether or not a crime drone has spawned which then attacks the current drone.

As there are multiple uses of the imported `random` class, it leads to no simulation being the same as the other, hence for better testing purposes, we have implemented a way to set a seed in the class called `RandomGeneratedNumbers`.

## 3.2 Use of Inheritance

We used inheritance for the structure of the drone objects, with *drone* as the super class, with two subclasses called *DeliveryDrone* and *CombatDrone*, these two then had two more subclasses each, with the *DeliveryDrone* subclasses being HighSpeedDrone and ArmoredDrone, and the subclasses for *CombatDrone* being GatlingDrone and EmpDrone.

The *Drone* class is designed to contain the fundamental variable all drones will need, such as a state, ID, and more. But more importantly than that, it contains a method that all drones need, which is the move method, that allows a drone to towards it's target location and takes care of changing the state of a drone once it has reached its location.

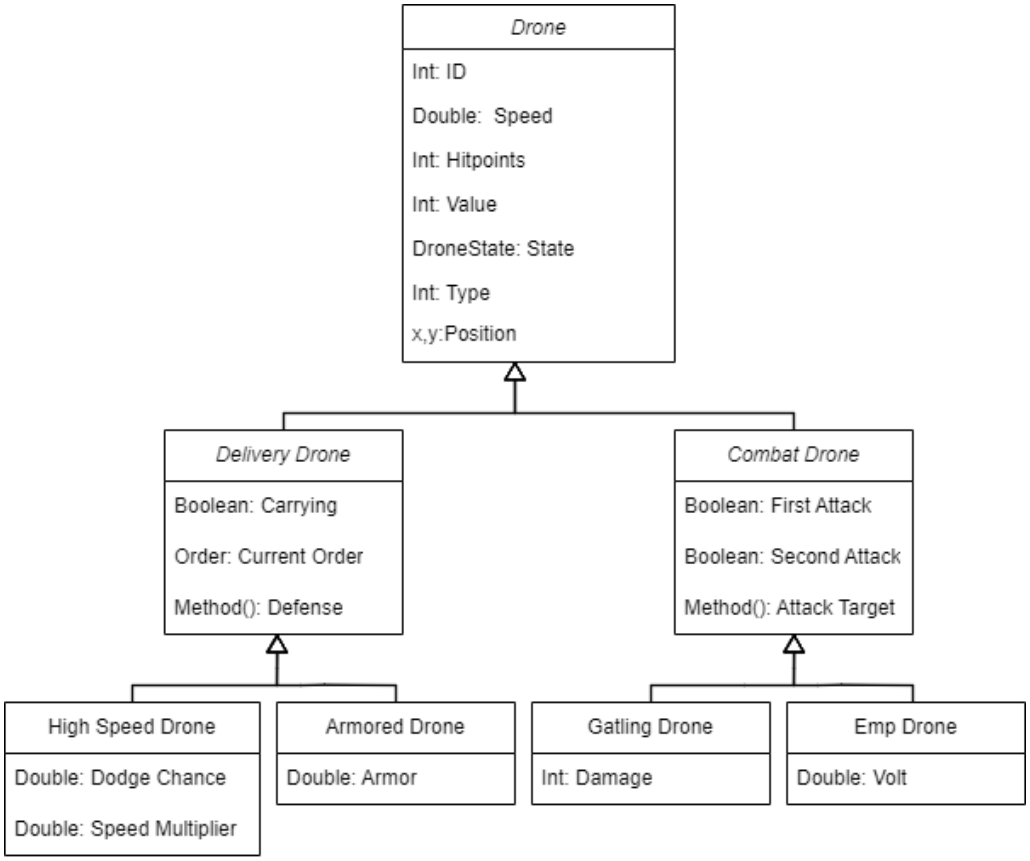


Figure 1: UML Diagram for Drone classes

The *Delivery Drone* class has some variables, the most notable ones being `Carrying` and `Current Order`, which are used to track if a drone has already picked up its item from a warehouse and what order it currently has assigned to it. Aside from this, it also contains an abstract method called *Defense*.

Within the subclasses High Speed Drone and Armored Drone, they contain variables pertaining to the inherited defense method and by use of polymorphism, the High Speed Drone gets a burst of speed whenever the defense method is called, allowing it to outrun the criminal drone attacking it, whilst the armored drone has a chance to ignore damage taken, with the chance depending on the armor variable declared within the subclass.

Both types of combat drones have also inherited a method called *AttackTarget*, and again by use of polymorphism, this method allows the Gatling Drone to attack a delivery drone and the EMP Drone has a chance to instantly neutralize it's targeted delivery drone, this chance being affected by the volt variable.

### 3.3 The crime objects

The `crime` class is where criminal combat drones are spawned from, by being passed a delivery drone, it decides which quadrant to spawn the combat drone and which delivery drone it is assigned to attack. The crime object is passed a list of stats for combat drones when created, and when it then has to create a combat drone, it uses the passed list to extract the stats, most of these stats are fixed, except for health and speed of the combat drone, which are randomly decided from the minimal and maximum value, which are also extracted from the list. This allows the simulation to never have two identical combat drones, leading to an even greater number of possible outcomes in the simulation.

Once the combat drones spawn and target a delivery drone, they will strike twice, if the drone is still alive after this, it leaves combat and goes back into transit, and the combat drone object will be removed.

## 4 Weaknesses of our solution

There are a few known weaknesses in our project, which could have been fixed with more times, most notably is it that *DeliveryDrone* has an abstract method named *speedburst*, this could have been part of the defense method instead, but in our current solution it is not, and it leads to `ArmoredDrone` having a speedburst method that simply returns a false.

There are some extensions lacking, most notably the one that lets you skip forward in time in intervals, currently our solution forces the user to hold the enter button to iterate quickly, leading to some less than ideal user friendliness.

There are some methods currently in the `Simulation` class, that could well be moved to `CombatDrone`, most notably the one called `strikeDeliveryDrone`, which makes all active combat drones attack their currently targeted delivery drone and the one called `spawnCrimeDrone`, could be moved into the crime class, as it spawns a combat drone in a quadrant decided by the variable in the crime object.

There is a fairly hard coded method in `Delivery Drone`, called `isAtWarehouse`, where we directly match the delivery drones coordinates to some numbers, which are identical to all the different warehouses coordinates, but are not part of their object. A solution to this would be to pass the list of warehouses to this method when called, and compare the delivery drone coordinates directly to the coordinates in the list instead.

The move method was declared in the `drone` class, when in reality it should have been in the `DeliveryDrone` class, since combat drones never utilize it, therefore they have no reason to inherit it. A quick solution to this is simply move the method to the `DeliveryDrone` class.

# 5 Conclusion

We managed to create a simulation following the problem statement given to us. We were able to implement extensions number 1, 4 and 6. Those being, giving the user the option to choose when to start the simulation and when it ends, delivery history of a specific drone at any point in the simulation and that drones can pass each other and providing a statistic for it at the end of the simulation.

Our solution uses polymorphism and inheritance in regards to the drones, generic ArrayLists to store objects and data, all our classes are cooperating and their responsibilities are distributed accordingly, with the simulation class tying it all together.

After some time of testing, we were unable to crash the program due to erroneous input, allowing us to conclude that the catch clauses should cover all inputs.

All of our important methods are black box tested by use of JUnit, allowing us to be fairly certain that they work as intended.

All relevant methods have a javadoc explaining them and their passed methods, we decided it was unnecessary for setters and getters, as we deemed it very obvious as to their function.